# Комп'ютерний зір для дрона Swift

https://gemini.google.com/app/2ba78167e86c3cb9

*User prompt: Я роблю ресерч щодо computer vision. Я розбираюсь в цій темі але мені треба пункти про що дійсно узфул ту телл. Тема: «Як навчити машину бачити: від симуляції до реального світу» (на прикладі дронів) Загальний CV покриють інші групи — ми йдемо вглиб. 🎯 Наскрізний приклад — дрон Swift (Цюрих, Nature 2023), що обігнав чемпіонів-людей у дрон-рейсингу. Кожен блок показує свою частину на ньому → презентація = одне ціле. ▶️ Вступ: де це вже працює — доставка, агро, рятувальні операції, війна. 1️⃣ Що потрібно моделі - архітектура: згорткові мережі (CNN) і т.д. - функція помилки (як модель розуміє, що помилилась), метрики - обчислення: тренування на GPU vs маленький чип на дроні (edge) 💬 дискусія: точність vs швидкість — велика модель точніша, але дрон не може чекати*

Response: Чудова база. Для наукового ресерчу та глибокої презентації важливо відійти від загальних визначень із підручників («CNN — це сітка для картинок») і дати інженерні інсайти, які вирішують реальні проблеми на швидкості понад 100 км/год.

Ось поглиблені тези для **Блоку 1**, прив'язані до наскрізного кейсу **Swift**:

## Блок 1: Що потрібно моделі (Глибинний розбір для Swift)

### 1. Архітектура: Чому «просто CNN» недостатньо

- **Latency Budget (Бюджет часу на кадр):** У дрон-рейсингу кадр надходить кожні 5–10 мс. Якщо нейромережа рахується 20 мс, дрон на швидкості летить у «сліпу» зону кілька метрів. Тому архітектури оптимізують не лише за точністю, а за *inter-frame latency*.
- **Легкі бекбони та гібриди:** Використовують не важкі ResNet чи ViT (Vision Transformers без важкої дистиляції занадто повільні для Edge), а мікроархітектури або гнучкі згорткові мережі, які видають максимальний FPS. Swift об'єднував сприйняття візуальних фіч (ворота) із політикою керування (Policy Network).

### 2. Функція помилки (Loss / Reward) у реальному світі

- **Від класифікації до RL/Imitation Learning:** Swift тренувався за допомогою глибокого підкріплювального навчання (Deep Reinforcement Learning) та клонування поведінки (Imitation Learning).
- **Комбінована функція нагород (Reward Function):** Це не просто Cross-Entropy. У нього входили:
    - Штраф за відхилення від траєкторії центру воріт.
    - Штраф за затримку часу проходження кола (Lap time minimization).
    - Плавність керування (Control smoothness), щоб дрон не «смикався» від мінімальних шумок у кадрі.

### 3. Обчислення: GPU Cluster vs. Edge Chip на борту

- **Training (GPU):** Мільйони симульованих годин на потужних кластерах NVIDIA GPUs (рендеринг ворожих трас, симуляція фізики, світла, аеродинаміки).
- **Inference (Edge):** Бортовий обчислювач дрона (наприклад, чипи класу NVIDIA Jetson / мобільні бортові NPUs) має жорсткі обмеження по вазі, енергоспоживанні (W) та тепловиділенню.
- **Квантування (Quantization):** Моделі з плаваючою комою (FP32) переводять у низьку розрядність (INT8 або FP16) за допомогою інструментів на кшталт NVIDIA TensorRT, щоб прискорити інференс у рази без втрати точності детегування вороть.

💬 **Ключова дискусія блоку: Точність vs Швидкість (The Speed-Accuracy Trade-off)**

- Велика модель бачить ворота ідеально, але її затримка (latency) призводить до розбиття дрона об стіну через запізнення реакції.
- **Висновок для Swift:** Краще мати злегка менш точну, але надшвидку нейромережу у поєднанні з пророкуванням траєкторії (Kalman Filter / State Estimation), ніж надточну «гальмувальну» архітектуру.

Переходимо до наступного етапу чи хочете додати щось специфічне до архітектури Swift (наприклад, як вони обходили проблему руху камер і Motion Blur)?

---

*User prompt: Поглибся в перший пункт. Я би хотів технічні шутки розповісти з частини ML. Які шари є і т.д. Які є різні технічні штуки з CV про які можна розповісти? CNN, kernel і т.д. Хочу розповісти технічно як з зображаня взагалі щось можна зрозуміти. Я буду пояснювати одногрупникам (1 курс) і в них немає контекту. Я хочу розповісти що існує умоно 3 типи CV: класифікація, детекція, сегментація. Ми розповідаєм саме про CV а дрон як приклад але на скільки я знаю дрон використовує завсім не один з цих 3 а щось інше для своєї роботи*

Response: Це чудовий інсайт і ти абсолютно правий щодо класичних типів CV! Давай розберемо це так, щоб першокурсникам було і зрозуміло, і захопливо, з крутими технічними метафорами.

## Частина 1. Твоя інтуїція залізобетонно правильна: чому дрон не робить класичне CV?

У навчальних курсах зазвичай виділяють **3 класичні задачі Computer Vision**:

1. **Класифікація (Classification):** «Що це на картинці?» (наприклад, лейбл *«кіт»* чи *«дрон»*).
2. **Детекція (Object Detection):** «Де це на картинці?» (малюємо рамку/bounding box навколо об'єкта: *«тут ворота, а тут інший дрон»*).
3. **Сегментація (Segmentation):** «Який піксель до чого належить?» (робимо маску на кожен об'єкт окремо або виділяємо лише дорогу/небо).

**Чому гоночний дрон (як Swift) цього НЕ робить?** Усі ці три класичні задачі видають **статичний результат** («картинка розпізнана»). Але дрон летить зі швидкістю понад 100 км/год. Якщо він витрачатиме час на те, щоб намалювати навколо вороток рамку (детекція) чи тим паче розфарбувати кожен піксель (сегментація), він вріжеться в стрічку ще до того, як кадр дорахується.

**Що робить дрон?** Він використовує **End-to-End Imitation / Reinforcement Learning (Скрізну полісі)**. Нейромережа на вході приймає сирі пікселі з камери (або вектор від датчиків), а на виході видає **не назву об'єкта**, а готові команди керування: *«тяга двигунів + кутова швидкість»* (thrust & body rates).

> *Технічний жарт для лекції:* «Класичне CV дивиться на світ і каже: "О, дивіться, це ворота!". Дрон у режимі реального часу дивиться на світ і каже: "Мені начхати, як це називається, я просто зараз крутану штурвал на 15 градусів ліворуч, щоб не розбитись"».

## Частина 2. Технічні штуки з CV «на пальцях» (Для першокурсників)

Щоб пояснити, як машина взагалі «розуміє» картинку, розкажи про це через базові математичні операції, які відбуваються всередині мережі.

### 1. Зображення для комп'ютера — це просто величезна таблиця (тензор)

- Для нас це красива фотографія вороток на трасі. Для комп'ютера кольорове зображення (RGB) — це три матриці (червона, зелена, синя канали) розміром, наприклад, 1920x1080, де кожна ячейка (піксель) — це число від 0 до 255 (яскравість).
- Машина не «бачить» красу — вона просто бачить цифри на кшталт `[[255, 120, 3], [45, 89, 200]...]`.

### 2. Ядро (Kernel / Фільтр) та згортка (Convolution) — магія перетворення

Як знайти на цьому полі цифр краї вороток, кути чи лінії?

- **Що таке ядро (Kernel):** Це маленька матриця (наприклад, 3x3 або 5x5 пікселів), яка бігає по всьому зображенню.
- **Як це працює:** Вона множить значення пікселів зображення на свої ваги і сумує їх (операція згортки).
- *Аналогія:* Уяви невелике сито або лінійку-трафарет, яку ти прикладаєш до різних частин картини. Якщо ядро налаштовано на пошук вертикальних ліній (а краї гоночних воріт якраз вертикальні), воно спрацьовує і видає високе число там, де лінія є, і нуль там, де просто однотонний фон.

  *Жартом:* «Згортка — це коли мережа бере маленьке віконечко, сканує картинку піксель за пікселем, і якщо бачить бордовий колір вороток на тлі сірої стіни, нейромережа каже: "О, тут пахне ганчіркою від воріт, їдемо далі!"».

### 3. Шари (Layers) мережі: від простого до складного

У глибоких мережах шари вишикувані послідовно, і кожен вирішує свою задачу:

- **Нижні шари (Early Layers):** Шукають найпростіші речі — краї, точки, градієнти, смужки.
- **Середні шари (Middle Layers):** Збирають ці палички та смужки у геометричні фігури (кути, кола, прямокутники воріт).
- **Глибокі шари (Deep Layers):** Розуміють контекст — «ага, ці чотири кути утворюють ворота, і вони праворуч під кутом 30 градусів».

## Як це зібрати в єдиний пазл для виступу:

1. **Почати з бази:** «Комп'ютер не бачить світ як ми — для нього фото це просто матриця з чисел від 0 до 255».
2. **Пояснити інструменти:** «Щоб витягнути з цієї матриці хоч якийсь сенс, придумали згортку (Convolution) та ядра (Kernels) — це такі маленькі матриці-фільтри, які бігають по фото і шукають лінії, плями та кути».
3. **Зробити місток до реальності та розбити міф:** «У підручниках вас вчитимуть про 3 класичні задачі CV: класифікацію, детекцію та сегментацію. Але якщо ви запустите класичний детектор об'єктів на гоночному дроні, який летить на швидкості 120 км/год, ваш дрон розіб'ється об першу ж стіну через затримку обчислень. Тому реальні передові системи (як Swift) викидають класичну детекцію на смітник і використовують пряме перетворення "пікселі $\rightarrow$ дії керування" за допомогою нейромереж та підкріплювального навчання».

**Sources**

1. Technical architecture of the Swift drone-racing system, including

---